



UNIVERSITY OF BUCHAREST

FACULTY OF
MATHEMATICS AND
COMPUTER SCIENCE



SPECIALIZATION: COMPUTER SCIENCE

Bachelor's Thesis

NEW APPROACHES TO TRANSLATING ONE PROGRAMMING LANGUAGE INTO ANOTHER: A CASE STUDY ON THE AVATAR DATASET

Graduate

Suhan Nicoleta Corina

Scientific Coordinator

Prof. dr. Ciprian Păduraru

Bucharest, June 2026

Rezumat

Această lucrare studiază traducerea automată Java-Python pe setul de date AVATAR, unde traducerile trebuie să păstreze comportamentul programului, nu doar forma codului. Sunt comparate patru soluții: zero-shot Qwen2.5-Coder-7B, UniXcoder encoder-only, Qwen2.5-Coder-0.5B cu QLoRA și CodeT5+ 220M Seq2Seq. Evaluarea folosește BLEU, CodeBLEU, verificare sintactică sau compilare, Execution Accuracy și Computational Accuracy. Cel mai bun sistem final este Qwen2.5-Coder-7B zero-shot, cu 59.68 Computational Accuracy pentru Java-to-Python și 61.80 pentru Python-to-Java. Dintre modelele antrenate, CodeT5+ 220M oferă cele mai bune rezultate funcționale.

Abstract

This thesis studies automatic Java-Python program translation on the AVATAR dataset, where translations must preserve program behavior, not only code form. Four solutions are compared: zero-shot Qwen2.5-Coder-7B, encoder-only UniXcoder, Qwen2.5-Coder-0.5B with QLoRA, and Seq2Seq CodeT5+ 220M. The evaluation uses BLEU, CodeBLEU, syntax checking or compilation, Execution Accuracy, and Computational Accuracy. The strongest final system is zero-shot Qwen2.5-Coder-7B, with 59.68 Computational Accuracy for Java-to-Python and 61.80 for Python-to-Java. Among the trained models, Seq2Seq CodeT5+ 220M gives the best functional results.

Contents

1	Introduction	5
2	Theoretical Background	6
2.1	Machine Translation	6
2.1.1	History	6
2.1.2	SoTA	7
2.1.3	Code translation	7
2.2	AVATAR Dataset	8
2.3	Evaluation Metrics	9
2.3.1	BLEU	9
2.3.2	Syntax Match	10
2.3.3	Dataflow Match	10
2.3.4	CodeBLEU	10
2.3.5	Execution Accuracy	11
2.3.6	Computational Accuracy	11
3	Methodology	12
3.1	Baseline	12
3.2	Zero-Shot Solution with Qwen2.5-Coder	12
3.3	Encoder-Only Solution with UniXcoder	13
3.4	Decoder-Only Solution with Qwen2.5-Coder	13
3.5	Seq2Seq Solution with CodeT5+	14
4	Experimental Setup	15
4.1	Zero-Shot Qwen2.5-Coder	15
4.2	Encoder-Only UniXcoder	15
4.3	Decoder-Only Qwen2.5-Coder	16
4.4	Seq2Seq CodeT5+	16
5	Results and Discussion	17
5.1	Reference Results from the AVATAR Paper	17

5.2	Results of the Implemented Models	18
5.3	Discussion	19
6	Conclusions and Future Work	20
6.1	Conclusions	20
6.2	Future Work	21
A	Prompts Used in the Zero-Shot Solution	22
A.1	System Prompts	22
A.2	Base User Prompt Template	22
A.3	Python Target Prompt Addition	23
A.4	Java Target Prompt Addition	24
	Bibliography	25

Chapter 1

Introduction

Machine translation evolved a lot in the last years, but my question was if the subtask of code translation evolved at the same level. Translating programs from one language to another can support code migration, reuse, and maintenance, but the task is harder than ordinary text translation. A correct program translation must preserve executable behavior: the generated code has to compile, run, and produce the same outputs as the source program.

This thesis studies Java-Python translation on the AVATAR dataset, which contains aligned competitive-programming solutions and execution test cases. The task is evaluated in both directions, Java-to-Python and Python-to-Java. The main challenge is that lexical similarity is not enough: a translation can obtain reasonable BLEU or CodeBLEU scores while still failing to preserve the required algorithmic behavior. For this reason, the evaluation also includes compilation or syntax checking, Execution Accuracy, and Computational Accuracy.

This work compares one modern model for each required category: zero-shot Qwen 7B, encoder-only UniXcoder with a learned Transformer decoder, decoder-only Qwen 0.5B with QLoRA, and Seq2Seq CodeT5+ 220M. The strongest final system is the zero-shot Qwen model, reaching 59.68 Computational Accuracy for Java-to-Python and 61.80 for Python-to-Java. Among the trained systems, the Seq2Seq CodeT5+ model gives the best functional result, reaching 47.82 Computational Accuracy for Java-to-Python and 41.29 for Python-to-Java after two epochs of fine-tuning per direction.

Compared with the original AVATAR baselines, this work focuses on modern pre-trained code models. The comparison shows the practical effect of code pretraining, instruction tuning, and parameter-efficient fine-tuning under the same benchmark and metric protocol. It also shows why execution-based evaluation is necessary: modern models can generate code that is textually close to the reference while still failing to preserve the program behavior.

Chapter 2

Theoretical Background

2.1 Machine Translation

2.1.1 History

Machine translation is one of the oldest goals of natural language processing, and its history is easiest to follow as a sequence of paradigms. Each paradigm gave a different answer to the same question: where should the knowledge needed for translation come from? Rule-based systems encoded it by hand, statistical systems estimated it from parallel data, and neural systems learn it end to end inside a single trainable model.

Rule-based machine translation dominated the early decades of the field. Systems from this period combined bilingual dictionaries, hand-written grammatical rules, and transfer procedures designed by linguists. The best-known representative is SYSTRAN, which was used for many years in real institutional and commercial settings. The strength of the rule-based approach was control: because every rule was written by a person, the behavior of the system could be inspected, explained, and corrected. Its weakness was cost: every new language pair and every new domain required additional expert work.

Statistical machine translation replaced hand-written rules with probabilities estimated from bilingual corpora: researchers collected aligned sentence pairs and let the system learn word and phrase correspondences from the data. The IBM alignment models introduced the mathematical core of this approach, and phrase-based systems later turned it into the dominant technology of the 2000s. Statistical systems clearly outperformed rule-based ones whenever enough parallel data existed, but they still translated through a pipeline of separately trained components, which limited how much context each decision could use.

Neural machine translation changed this picture by learning the entire mapping from the source sequence to the target sequence in one end-to-end model. In the early neural period, the standard architecture was an encoder-decoder built from recurrent networks. The system that best marked the maturity of this paradigm was Google’s Neural Machine

Translation system, GNMT [17], which showed at production scale that neural models could overtake strong phrase-based baselines.

The current period began with the Transformer architecture [14]. By replacing recurrence with self-attention, the Transformer trains efficiently in parallel and models long-distance dependencies directly, because any token can attend to any other token in the sequence. These properties quickly made it the dominant architecture in machine translation.

2.1.2 SoTA

In 2026, the state of the art in machine translation is no longer tied to a single model name. Different systems lead in different settings, and the strongest results come from a family of very large multilingual Transformer-based models, often combined with LLM-style adaptation or prompting strategies. A good reference point for this direction is the No Language Left Behind project and its NLLB-200 model, which showed that one multilingual system can cover around two hundred languages while keeping strong quality, including for many low-resource pairs [13].

Recent shared-task results show that the field has entered a more mixed phase. The WMT24 General Machine Translation task makes two points at the same time: large language models are now central to machine translation, and the task is still not solved [3]. Very strong results are possible, but they depend on the language pair, the domain, the model size, the adaptation strategy, and the evaluation protocol. No single system wins in every setting.

The way quality is judged has also become broader: beyond automatic scores on one benchmark, researchers look at multilingual robustness, domain transfer, stability on low-resource languages, and the gap between automatic metrics and real usefulness. Modern machine translation research is therefore less about finding one new architecture and more about combining strong pretrained models, multilingual data, and careful tuning and evaluation.

2.1.3 Code translation

Code translation applies the machine translation idea to programming languages, but the two tasks differ in one essential point: the standard of correctness. A natural language sentence usually has several acceptable translations, and small variations in word choice rarely make the output wrong. A program is judged much more strictly. A translated program that does not compile, crashes at runtime, or computes a different result is not an imperfect translation but a wrong one, no matter how similar it looks to the reference.

The structure of the data is also different. Natural language tolerates ambiguity, redundancy, and stylistic variation. Programming languages are formal: a translation

must preserve the semantics of the source program while respecting the syntax, the scoping rules, the type system, the standard library, and the conventions of the target language. A single changed identifier or a shifted loop bound can silently change the behavior of an otherwise plausible program. Code translation is therefore not only sequence generation; the model must also track how values flow through the program.

The research history of code translation is shorter than that of general machine translation. Early systems were closer to transpilers and compiler engineering than to learned models: they worked well when the mapping between the two languages could be described by rules, but scaled poorly once the correspondence became less direct.

The modern phase started when pretrained neural models for source code became available. An important milestone was TransCoder, which showed that translation between programming languages is possible even without parallel programs, using unsupervised training [12]. Pretrained encoder-decoder models such as PLBART then showed that code generation and translation benefit from pretraining on large amounts of source code and technical natural language [2]. The overall direction mirrors what happened in general machine translation, with one important difference: in code translation, evaluation must go beyond text similarity, because a translation is useful only if it preserves the behavior of the original program.

2.2 AVATAR Dataset

AVATAR is the central dataset of this thesis. It was built specifically for program translation between Java and Python [1]. Unlike the large code corpora used for pretraining, AVATAR is a parallel corpus: for each programming problem it contains a Java solution and a Python solution that solve the same task. This alignment provides the direct source-target relation needed for supervised translation experiments and for a controlled comparison between models.

The examples come from competitive-programming platforms and similar sources, so each program is short, self-contained, and focused on a single algorithmic idea. This format suits translation research well: each pair shows how the same algorithm is expressed in the idioms of the two languages, without the project-level dependencies of full software repositories. The dataset comes with a fixed train, validation, and test split, and all models in this thesis are trained and evaluated on the same split, so the reported differences can be attributed to the models rather than to the data.

The property that matters most for this thesis is that AVATAR connects each problem to input-output test cases. Text similarity alone cannot establish whether a translation is correct: a generated program can be close to the reference at the token level and still fail to compile or print a wrong answer. The test cases make it possible to compile and execute every generated translation and to check its actual behavior, and they are the

basis of the execution-based metrics used in this work.

AVATAR also has clear limitations. It covers a single language pair, and its programs reflect competitive-programming style rather than production software, so its results do not automatically transfer to every code translation scenario. Within these limits, it remains one of the most useful resources for controlled Java-Python translation experiments.

2.3 Evaluation Metrics

The evaluation in this thesis uses two families of metrics. The first family measures how similar the generated program is to the reference translation, as text or as structure: BLEU, Syntax Match, Dataflow Match, and CodeBLEU. The second family ignores the reference text and asks what the generated program actually does when it is compiled and executed: Execution Accuracy and Computational Accuracy. The two families answer different questions, and the experiments in this thesis show that they can disagree strongly: a system can produce code that looks very close to the reference and still fails the tests. For this reason, every model is reported on both families.

2.3.1 BLEU

BLEU is the classical metric of machine translation [8]. It compares the candidate translation with the reference at the level of n -grams, which are sequences of n consecutive tokens. For each order n from 1 to 4, BLEU computes a modified precision p_n : the fraction of candidate n -grams that also appear in the reference, where each reference n -gram can be matched at most as many times as it occurs in the reference. The four precisions are combined by a geometric mean, and the result is multiplied by a brevity penalty BP that lowers the score when the candidate is shorter than the reference:

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^4 \frac{1}{4} \log p_n \right), \quad \text{BP} = \min \left(1, e^{1-r/c} \right),$$

where r is the total reference length and c is the total candidate length. In the convention used here, the score is scaled to the range from 0 to 100 and computed over the whole test corpus rather than per example.

For code, BLEU is applied to the token-spaced representation of the programs, so it measures how many token patterns the translation shares with the reference solution. It is a useful metric, but it does not reveal the whole information about translation. However, BLEU has no notion of program structure or meaning. Renaming one variable lowers the score without changing the behavior, while replacing `<` with `<=`, which can break the program, costs almost nothing.

2.3.2 Syntax Match

Syntax Match (SM) is the structural component of CodeBLEU [11]. Both the candidate and the reference program are parsed into abstract syntax trees, the trees are decomposed into subtrees, and the score is the fraction of reference subtrees that also occur in the candidate tree. Intuitively, it asks whether the translation is built from the same syntactic constructions as the reference: the same loop nesting, the same conditional shapes, the same expression patterns.

The motivation is that source code is hierarchical, not linear. Two programs can share most of their tokens and still have different structure, and the structure carries meaning: moving a statement into or out of a loop body changes the behavior completely while changing very little at the token level. Syntax Match captures this kind of agreement better than n-gram overlap. Its limitation is that it still compares against a single reference, so a correct program written with a different structure, for example a `while` loop instead of a `for` loop, is penalized even when its behavior is identical.

2.3.3 Dataflow Match

Dataflow Match (DM) is the semantic component of CodeBLEU [11]. From each program, a data-flow graph is extracted: the nodes are variables and the edges record where each value comes from, for example which assignment defines the variable that a later expression reads. The score is the fraction of reference data-flow edges that are matched in the candidate program.

This metric looks one level deeper than syntax. A translation can use the right constructions in the right order and still connect the wrong values, for instance by reading a loop counter where the accumulated sum was intended. Such a mistake barely affects BLEU or Syntax Match, but it breaks the dependency structure that Dataflow Match checks. Because variable names are abstracted away during matching, the metric is also robust to renaming, which plain token overlap is not. Among the text-based metrics it is the closest to semantics, although it still cannot confirm that the program computes the correct result.

2.3.4 CodeBLEU

CodeBLEU combines the previous signals into one score designed for code [11]. It is a weighted sum of four components:

$$\text{CodeBLEU} = \alpha \cdot \text{BLEU} + \beta \cdot \text{BLEU}_w + \gamma \cdot \text{SM} + \delta \cdot \text{DM},$$

where BLEU_w is a weighted n-gram match that gives the keywords of the language more importance than ordinary tokens. In the standard configuration used in this thesis, the

four weights are equal, $\alpha = \beta = \gamma = \delta = 0.25$.

The intent is to judge code the way a reviewer would: partly by its tokens, partly by its structure, and partly by how data moves through it. CodeBLEU is usually more informative than plain BLEU for code tasks. The averaging also has a side effect that matters when reading result tables: a weak component can be hidden by strong ones, so the individual components are reported alongside the combined value.

2.3.5 Execution Accuracy

Execution Accuracy (EA) is the first execution-based metric [1]. Every generated translation is converted back into normal source code and actually run on the test inputs of its problem. A program counts as executable when it passes the language processing stage, which is compilation for Java and a syntax check for Python, and then runs on the test inputs without raising a runtime error. Execution Accuracy is the percentage of test problems whose translation is executable in this sense. Programs that run but print a wrong answer still count, because this metric measures only the ability to run.

Execution Accuracy is a much stricter filter than any similarity score, because it is applied by the compiler and the runtime rather than by comparison with a reference. It catches missing declarations, type errors, syntax mistakes, and crashes, which are exactly the failures that text metrics tend to overlook. Its limit is that it says nothing about what the program computes, so a high Execution Accuracy alone does not mean that the translations are correct.

2.3.6 Computational Accuracy

Computational Accuracy (CA) is the strictest metric in this thesis and its main criterion for comparing models. A translation is counted as correct only when it compiles, runs on every available test case of its problem, and produces the expected output on all of them; numeric outputs are compared with a small tolerance. The metric is the percentage of test problems for which this happens. The same idea was introduced for code translation by TransCoder [12] and is used by the AVATAR benchmark [1].

Computational Accuracy is the closest automatic proxy to the real goal of the task: a translated program that behaves exactly like the source program. It subsumes the other metrics in one direction only: a program that passes all tests usually also has reasonable similarity scores, but high BLEU or CodeBLEU values do not imply high Computational Accuracy. This gap is a recurring observation in the results of this thesis, and it is the main reason why execution-based evaluation is treated as mandatory for code translation.

Chapter 3

Methodology

3.1 Baseline

The reference baseline for this project is the trained-from-scratch Transformer used in the AVATAR paper [1]. It is kept as a comparison point because it uses the same benchmark and the same translation directions, but it does not benefit from code pretraining or instruction tuning. The model is a standard encoder-decoder Transformer [14]: the encoder reads the token-spaced source program, the decoder generates the token-spaced target program, and self-attention allows both sides to model dependencies between distant tokens.

The local baseline script follows this architecture with six encoder layers, six decoder layers, shared token embeddings, AdamW optimization, label smoothing, and beam search. Its role in the thesis is not to be the main system, but to show the starting point provided by a supervised neural translator trained only on AVATAR. This makes the later pretrained solutions easier to interpret: any improvement can be related to the effect of pretraining, prompting, or parameter-efficient fine-tuning rather than only to the dataset itself.

3.2 Zero-Shot Solution with Qwen2.5-Coder

The first implemented solution is a zero-shot translator based on Qwen2.5-Coder-7B-Instruct. Qwen2.5-Coder is a family of code-oriented large language models trained for code generation, reasoning, and repair [7]. The selected checkpoint has 7.61 billion parameters [10]. This setting is called zero-shot because the model is not fine-tuned on AVATAR and the prompt contains no worked examples from the benchmark.

For each test program, the script sends the token-spaced source program to the chat model with a direction-specific instruction. The prompt asks the model to reconstruct the source program internally and return complete normal source code in the target language.

Additional rules discourage source-language leakage: Java-to-Python prompts reject constructs such as `Scanner` and `System.out.println`, while Python-to-Java prompts reject constructs such as `def`, `range`, and Python exponentiation syntax. Decoding is deterministic, with sampling disabled, so the same input and checkpoint produce the same translation.

The zero-shot pipeline deliberately separates generation from AVATAR formatting. Early tests showed that directly asking Qwen to generate the dataset token stream was less reliable than asking for normal code. Therefore, the final script first generates ordinary Python or Java code, then applies a deterministic tokenizer that converts the output back into AVATAR’s token-spaced representation. This design tests the ability of a modern instruction-tuned model to perform code translation without supervised benchmark training. The full prompt templates are included in Appendix A.

3.3 Encoder-Only Solution with UniXcoder

The second solution studies an encoder-only pretrained model. It uses `microsoft/unixcoder-base` [5] as the source-side encoder and adds a newly trained Transformer decoder for generation. This design answers a specific question: how much does a pretrained code representation help if only the encoder is reused and the target-language generator is learned from AVATAR?

The UniXcoder encoder receives the token-spaced source program and produces contextual source states. The added decoder is a four-layer Transformer decoder with causal self-attention, cross-attention over the encoder states, learned target positions, and an output projection tied to the encoder token embeddings. During training, the target program is shifted in the usual teacher-forcing setup: the decoder sees the previous target tokens and learns to predict the next one. During generation, it performs greedy autoregressive decoding until the end token or the maximum target length is reached.

This approach is intentionally asymmetric. The source side starts from a pretrained code model, while the target side has to learn syntax, formatting, and translation behavior from the parallel AVATAR examples. It is therefore a useful middle point between the scratch Transformer baseline and fully pretrained encoder-decoder models. If the encoder-only model performs well, the source representation alone is carrying much of the translation signal; if it performs weakly on execution metrics, the missing pretrained decoder is a likely limitation.

3.4 Decoder-Only Solution with Qwen2.5-Coder

The third solution fine-tunes a decoder-only instruction model: Qwen2.5-Coder-0.5B-Instruct [9]. Unlike the zero-shot system, this model is trained on AVATAR examples.

Unlike the encoder-only system, it has no separate encoder; translation is represented as conditional language modeling over a prompt and a target token stream.

Each training example is converted into a chat-style prompt that asks for a translation from token-spaced source code into token-spaced target code. The prompt tokens and the target tokens are concatenated into one causal sequence. The loss is applied only to the target part, so the model learns to continue the prompt with the correct translation rather than to reproduce the prompt itself.

The implementation uses a QLoRA-style setup [4]: the base model is loaded in 4-bit precision, and only small LoRA adapters [6] are trained on selected attention projection matrices. This makes the decoder-only experiment practical on Kaggle hardware while still testing whether an instruction-tuned causal code model can adapt to AVATAR’s exact token representation. The method also exposes a tradeoff: the model already has strong code knowledge, but the causal prompt format must fit both the source program and the target program inside one context window.

3.5 Seq2Seq Solution with CodeT5+

The fourth solution is a pretrained encoder-decoder translator based on CodeT5+ 220M [15]. This model is closest to the classical neural machine translation formulation used by the baseline, because one side encodes the source program and the other side decodes the target program. The main difference is that both sides start from code pretraining rather than from random initialization.

The selected checkpoint, `Salesforce/codet5p-220m`, is loaded through the Hugging Face `AutoModelForSeq2SeqLM` interface [16]. The encoder receives the AVATAR token-spaced source program, and the decoder is trained to generate the token-spaced target program directly.

The method uses the standard conditional generation objective for encoder-decoder models. During training, the encoder input is the source token sequence and the decoder target is the shifted target token sequence. During generation, the encoder states remain fixed while the decoder produces the translation autoregressively. This keeps the translation task explicit: the source program and target program are represented as two separate sequences, instead of being packed into one causal prompt.

Chapter 4

Experimental Setup

4.1 Zero-Shot Qwen2.5-Coder

The zero-shot experiment is implemented in `zeroshot_qwen_kaggle.py`. It uses `Qwen/Qwen2.5-Coder-7B-Instruct` with the Hugging Face causal language model interface [16]. The model is loaded on one GPU with 4-bit NF4 quantization, half-precision compute, remote model code enabled, and left-padded tokenization. No AVATAR training is performed. The script evaluates the full test split in both directions, Java-to-Python and Python-to-Java.

The run configuration disables sampling, uses one decoding beam, sets the temperature to zero, keeps the repetition penalty at 1.0, and allows up to 1024 new tokens. The batch size is 12 prompts and the seed is 0.

4.2 Encoder-Only UniXcoder

The encoder-only experiment is implemented in `encoder_only_unixcoder_kaggle.py`. It uses `microsoft/unixcoder-base` [5] as a pretrained source encoder and adds a four-layer Transformer decoder. The encoder remains trainable, encoder gradient checkpointing is enabled, and the decoder uses dropout 0.1. The output layer is tied to the encoder token embedding matrix.

Both directions are trained in the order Python-to-Java and Java-to-Python. The full train, validation, and test splits are used. Training uses three epochs, batch size 4, gradient accumulation over 4 steps, maximum source and target lengths of 512 tokens, label smoothing 0.1, weight decay 0.01, warmup ratio 0.06, gradient clipping at 1.0, and seed 1234. The encoder learning rate is 2×10^{-5} , while the decoder learning rate is 1×10^{-4} .

4.3 Decoder-Only Qwen2.5-Coder

The decoder-only experiment is implemented in `decoder_only_qwen_kaggle.py`. It uses `Qwen/Qwen2.5-Coder-0.5B-Instruct` [9]. The model is loaded in 4-bit NF4 precision with `float16` compute and fine-tuned with LoRA adapters [6, 4]. The LoRA rank is 4, $\alpha = 8$, dropout is 0.05, and the target modules are `q_proj` and `v_proj`. Gradient checkpointing is disabled for this run.

The script trains Python-to-Java and Java-to-Python. The run lasts one epoch with batch size 4, gradient accumulation over 4 steps, learning rate 3×10^{-4} , no weight decay, warmup ratio 0.06, gradient clipping at 1.0, two dataloader workers, prefetch factor 2, and seed 1234. Source and target lengths are capped at 320 tokens, with a maximum causal sequence length of 640 tokens.

4.4 Seq2Seq CodeT5+

The Seq2Seq experiment is implemented in `seq2seq_codet5p_kaggle.py`. It uses `Salesforce/codet5p-220m`. The model is loaded with the Hugging Face sequence-to-sequence interface [16]; the fast tokenizer is preferred, and the script falls back to a local byte-level BPE tokenizer if the fast tokenizer fails validation.

The task order is Java-to-Python followed by Python-to-Java, and the full AVATAR splits are used. Training uses two epochs, batch size 4, gradient accumulation over 4 steps, maximum source and target lengths of 512 tokens, learning rate 5×10^{-5} , AdamW weight decay 0.01, warmup ratio 0.06, gradient clipping at 1.0, automatic mixed precision enabled, gradient checkpointing disabled, two dataloader workers, and seed 1234.

Chapter 5

Results and Discussion

5.1 Reference Results from the AVATAR Paper

The original AVATAR paper evaluates several neural models on Java-Python program translation [1]. Table 5.1 reproduces the most useful columns from the paper’s program-translation results: BLEU for token-level similarity, CodeBLEU for code-aware similarity, and Computational Accuracy for functional correctness. The full paper table also reports Syntax Match, Dataflow Match, and Execution Accuracy.

Table 5.1: Reference results reported in the original AVATAR paper.

Model	Java-to-Python			Python-to-Java		
	BLEU	CB	CA	BLEU	CB	CA
TransCoder	38.7	36.4	0.0	45.2	32.4	0.0
DOBF	42.0	38.9	0.0	42.3	31.2	0.0
TransCoder-ST (zero-shot)	41.7	39.3	0.0	42.5	30.7	0.0
Seq2Seq + Attention	57.4	42.6	2.8	59.5	43.0	0.8
Transformer	39.6	34.8	0.4	43.5	35.6	0.4
CodeGPT	46.3	30.2	2.8	48.9	38.0	2.0
CodeGPT-adapted	44.3	29.3	2.4	48.0	36.7	0.8
CodeBERT	51.1	35.0	0.4	35.1	33.2	0.0
GraphCodeBERT	57.9	39.0	2.0	38.3	36.9	0.0
PLBART	63.1	46.2	6.8	69.7	48.8	0.8
CodeT5	62.7	46.2	6.0	60.8	50.3	1.6
TransCoder-ST (fine-tuned)	55.4	43.8	5.6	66.0	48.6	2.0

The reference results show why execution-based evaluation is necessary for this task. Several models obtain BLEU scores above 55 and CodeBLEU scores above 40, but their Computational Accuracy remains low. The best Java-to-Python value in the paper’s program-translation table is 6.8, obtained by PLBART, while the best Python-to-Java

value is 2.0, obtained by CodeGPT and the fine-tuned TransCoder-ST system.

5.2 Results of the Implemented Models

Table 5.2 summarizes the four implemented model categories. The same compact metric set is used as in Table 5.1, so the thesis results can be compared directly with the reference systems from the AVATAR paper.

Table 5.2: Results of the implemented models on the AVATAR test split.

Model	Java-to-Python			Python-to-Java		
	BLEU	CB	CA	BLEU	CB	CA
Zero-shot Qwen2.5-Coder 7B	58.23	45.67	59.68	61.02	48.51	61.80
Encoder-only UniXcoder	55.20	44.20	8.00	49.44	38.18	1.95
Decoder-only Qwen2.5-Coder 0.5B	55.58	36.49	24.68	46.59	37.69	11.34
Seq2Seq CodeT5+ 220M	73.11	47.37	47.82	72.07	51.76	41.29

The strongest implemented model by Computational Accuracy is the 7B zero-shot Qwen system. It reaches 59.68 for Java-to-Python and 61.80 for Python-to-Java without AVATAR-specific training. Compared with the AVATAR paper reference results, it is far above every program-translation model in Table 5.1 on functional correctness. Its BLEU and CodeBLEU are not the highest implemented values, which shows that surface similarity is not the only signal that matters for executable translation.

Seq2Seq CodeT5+ 220M is the strongest implemented fine-tuned model. It obtains the highest BLEU scores among the implemented systems, 73.11 and 72.07, and the highest implemented CodeBLEU score for Python-to-Java, 51.76. Its Computational Accuracy is also strong, with 47.82 for Java-to-Python and 41.29 for Python-to-Java. This places it well above the CodeT5 and PLBART reference results from the AVATAR paper on functional correctness, although still below the zero-shot Qwen system used in this thesis.

The decoder-only Qwen2.5-Coder 0.5B model sits between the strong fine-tuned encoder-decoder model and the weaker encoder-only experiment. Its Computational Accuracy values, 24.68 and 11.34, are above the reference range reported in the AVATAR paper’s program-translation table, but they do not reach the CodeT5+ results. This suggests that the causal model can learn useful translation behavior after fine-tuning, but the smaller checkpoint and the single-stream prompt format limit functional correctness.

The encoder-only UniXcoder model has reasonable lexical and CodeBLEU scores, but weak functional results. In Java-to-Python, it reaches 55.20 BLEU and 44.20 CodeBLEU, close to several reference models from the AVATAR paper, yet its Computational Accuracy is only 8.00. In Python-to-Java, the gap is larger: 49.44 BLEU and 38.18 CodeBLEU correspond to only 1.95 Computational Accuracy. This is near the top of the original

paper’s Python-to-Java functional range, but it remains weak in absolute terms. The result confirms that a pretrained source encoder is not enough by itself; the target-side generator is critical for producing behaviorally correct programs.

5.3 Discussion

The main pattern across both tables is the separation between textual similarity and functional correctness. BLEU and CodeBLEU reward programs that look close to the reference, but AVATAR requires preserving runtime behavior. This distinction explains why models such as PLBART, CodeT5, and GraphCodeBERT can have respectable BLEU and CodeBLEU values while still producing low Computational Accuracy in the original paper.

The implemented results follow the same pattern. CodeT5+ is the best model for surface similarity, but zero-shot Qwen2.5-Coder 7B is the best model for functional correctness. This difference is important: the larger instruction model does not reproduce the reference format as closely as the fine-tuned Seq2Seq model, but it more often preserves the behavior of the source program. For a program translation system, this is the more important property.

The comparison with the AVATAR paper also shows that model scale and pretraining objective matter. The zero-shot Qwen model exceeds the reference systems on functional correctness even without supervised training on the benchmark, while the smaller decoder-only model improves over the original functional range but does not reach the same level. The encoder-only model gives the clearest negative result: reusing only a pretrained encoder helps produce plausible text, but it does not solve the harder problem of generating correct target-language programs.

Overall, the best result of this thesis is not the model with the highest BLEU, but the model with the highest Computational Accuracy. The results therefore support the central evaluation choice of the project: code translation should be judged by behavioral correctness, with lexical and structural metrics used as supporting evidence rather than as the final criterion.

Chapter 6

Conclusions and Future Work

6.1 Conclusions

This thesis studied Java-Python program translation on the AVATAR benchmark, with an emphasis on whether modern pretrained code models can produce translations that are not only textually similar to the reference, but also functionally correct. The work compared four model categories: a zero-shot instruction model, an encoder-only model, a decoder-only model, and a Seq2Seq encoder-decoder model. This organization made it possible to compare different ways of using pretraining: prompting a large model directly, reusing only a source encoder, adapting a causal decoder with parameter-efficient fine-tuning, and fine-tuning a pretrained encoder-decoder model.

The results show that Computational Accuracy is the most important metric for this task. BLEU and CodeBLEU are useful because they measure token-level and code-aware similarity, but they do not guarantee that the translated program preserves the behavior of the source program. This was visible both in the original AVATAR paper and in the experiments from this thesis. Several models produced translations that looked close to the reference, but only a smaller part of those translations passed the available execution tests.

The strongest implemented system was the zero-shot Qwen2.5-Coder 7B model. It reached the highest Computational Accuracy in both translation directions, with 59.68 for Java-to-Python and 61.80 for Python-to-Java, even though it was not fine-tuned on AVATAR. This result shows the strength of recent instruction-tuned code models: a sufficiently capable model can solve many translation examples from prompting alone. The Seq2Seq CodeT5+ 220M model was the strongest fine-tuned system. It achieved the highest BLEU scores among the implemented models and strong Computational Accuracy values of 47.82 and 41.29. This confirms that pretrained encoder-decoder models are a strong fit for supervised code translation.

The other two model categories were useful because they clarified the limits of partial

adaptation. The decoder-only Qwen2.5-Coder 0.5B model improved over the weaker reference results from the original AVATAR paper, but it remained below the larger zero-shot model and the Seq2Seq CodeT5+ model. The encoder-only UniXcoder model produced reasonable lexical similarity, but its functional correctness remained low. This shows that source-side pretraining alone is not enough for reliable program translation; the target-side generator also needs enough capacity and task adaptation.

All experiments were implemented and run in a reproducible project structure. Kaggle was used as the main execution environment because it provided access to GPU resources and made it possible to train and evaluate models without a local high-end machine. The complete code for the project is available at <https://github.com/nicosuhan/avatar>.

6.2 Future Work

The main limitation of this work was computational budget. Because the experiments were run on limited Kaggle resources, there was not enough time to train every model for more epochs or to run a wider hyperparameter search. Future work should therefore repeat the supervised experiments with longer training schedules, especially for the decoder-only and Seq2Seq models. More epochs could help the models adapt better to AVATAR’s token-spaced format and could reduce the gap between lexical similarity and functional correctness.

Another direction is to test larger checkpoints under the same evaluation protocol. The zero-shot 7B model performed very well, while the fine-tuned 0.5B decoder-only model was much weaker. This suggests that model scale has a large effect on functional translation quality. Future experiments could fine-tune larger decoder-only models with LoRA or QLoRA [6, 4] and compare them directly with the zero-shot setup.

Finally, future work could improve the evaluation and error analysis. The current results show how many programs pass the available tests, but a deeper analysis could separate syntax errors, API mismatches, input-output mistakes, and algorithmic errors. This would make it easier to understand why a model fails and which architecture is best suited for each type of translation problem. Together with longer training and larger models, this would provide a clearer path toward more reliable Java-Python program translation.

Appendix A

Prompts Used in the Zero-Shot Solution

A.1 System Prompts

Java-to-Python:

You translate competitive-programming programs from token-spaced java into valid python. Return only python source code.

Python-to-Java:

You translate competitive-programming programs from token-spaced python into valid java. Return only java source code.

A.2 Base User Prompt Template

Translate this token-spaced {src_lang} program into normal {tgt_lang} source code.

Important:

- Reconstruct the source program before translating it.
- NEW_LINE means a line break. INDENT and DEDENT describe Python-style block nesting in the source.
- Preserve the algorithm, constants, input/output behavior, and edge cases.
- Write complete runnable {tgt_lang} code.
- Do not copy {src_lang} libraries, methods, wrappers, or syntax into the answer.
- Output normal {tgt_lang} source code, not a token stream.

- Keep hardcoded values hardcoded. Only read input when the source program reads input.
- Mentally check the final code for valid {tgt_lang} syntax before returning it.

Target language: {tgt_lang}
{target_examples}

Return only the translated {tgt_lang} source code. Do not explain. Do not use Markdown.

Source tokenized {src_lang} program:
{code}

A.3 Python Target Prompt Addition

Python output requirements:

- Output Python code only.
- Do not use Java syntax such as public, static, void, class Main, Scanner, String[] args, System.out.println, System.out.printf, charAt, toCharArray, or semicolons as statement endings.
- Use Python input(), sys.stdin, print(), sys.stdout.write(), lists, strings, math, and normal Python functions/classes where appropriate.
- Translate Java string methods to Python: s.charAt(i) becomes s[i], s.length() becomes len(s), toCharArray() becomes the same string or list(s).
- Translate Java Math methods to Python math functions or operators.
- If Java uses geometry/library objects, reimplement the math with Python functions, tuples, or complex numbers. Do not call methods like intersects(), distance(), p1, or p2 on tuples.
- Avoid shadowing Python builtins such as len, str, input, sum, max, min, list, dict, and set.

A.4 Java Target Prompt Addition

Java output requirements:

- Output Java code only.
- Do not use Python syntax such as `def`, `print()`, `range()`, `while True`, `import sys`, list methods, string `zfill`, or colon-based blocks.
- Use Java `Scanner/BufferedReader/StringTokenizer` only when the source reads input. Keep hardcoded test values hardcoded.
- Use `System.out.print/println/printf`, arrays, `ArrayList`, `StringBuilder`, `Math`, braces, and semicolons where appropriate.
- Translate Python `**` to Java `Math.pow(...)` or direct multiplication; never output `* *`.
- Translate Python `zfill(n)` to zero padding with `String.format(...).replace(' ', '0')` or equivalent Java code.
- Initialize boolean sieve arrays correctly, usually with `Arrays.fill(prime, true)`, before marking composites.
- Methods called from main should be static unless they are called through an object.

Bibliography

- [1] W. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang. “AVATAR: A Parallel Corpus for Java-Python Program Translation.” In: *Findings of the Association for Computational Linguistics: ACL 2023*. Association for Computational Linguistics, 2023, pp. 2281–2298.
- [2] W. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang. “PLBART: Unified Pre-training for Program and Language Understanding and Generation.” In: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2021, pp. 4173–4186.
- [3] M. R. Costa-jussà, C. Federmann, T. Kocmi, F. Guzmán, et al. “Findings of the WMT24 General Machine Translation Shared Task: The LLM Era Is Here but MT Is Not Solved Yet.” In: *Proceedings of the Ninth Conference on Machine Translation*. Association for Computational Linguistics, 2024.
- [4] T. Detrmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. “QLoRA: Efficient Finetuning of Quantized LLMs.” In: *Advances in Neural Information Processing Systems 36*. Curran Associates, Inc., 2023.
- [5] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin. “UniXcoder: Unified Cross-Modal Pre-training for Code Representation.” In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2022, pp. 7212–7225.
- [6] E. J. Hu, Y. Shen, P. Wallis, et al. “LoRA: Low-Rank Adaptation of Large Language Models.” In: *International Conference on Learning Representations*. 2022.
- [7] B. Hui, J. Yang, Z. Cui, et al. “Qwen2.5-Coder Technical Report.” In: *arXiv preprint arXiv:2409.12186* (2024).
- [8] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu. “BLEU: a Method for Automatic Evaluation of Machine Translation.” In: *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2002, pp. 311–318.
- [9] Qwen Team. *Qwen2.5-Coder-0.5B-Instruct*. <https://huggingface.co/Qwen/Qwen2.5-Coder-0.5B-Instruct>. Accessed: 2026-06-12. 2024.
- [10] Qwen Team. *Qwen2.5-Coder-7B-Instruct*. <https://huggingface.co/Qwen/Qwen2.5-Coder-7B-Instruct>. Accessed: 2026-05-12. 2024.

- [11] S. Ren, D. Guo, S. Lu, et al. “CodeBLEU: a Method for Automatic Evaluation of Code Synthesis.” In: *arXiv preprint arXiv:2009.10297* (2020).
- [12] B. Roziere, M.-A. Lachaux, L. Chansussot, and G. Lample. “Unsupervised Translation of Programming Languages.” In: *Advances in Neural Information Processing Systems 33*. Curran Associates, Inc., 2020.
- [13] T. N. Team, M. R. Costa-jussà, O. Celebi, et al. “Scaling Neural Machine Translation to 200 Languages.” In: *Nature* 630.8015 (2024), pp. 841–846. DOI: [10.1038/s41586-024-07335-x](https://doi.org/10.1038/s41586-024-07335-x).
- [14] A. Vaswani, N. Shazeer, N. Parmar, et al. “Attention Is All You Need.” In: *Advances in Neural Information Processing Systems 30*. Curran Associates, Inc., 2017.
- [15] Y. Wang, H. Le, A. D. Gotmare, N. D. Q. Bui, J. Li, and S. C. H. Hoi. “CodeT5+: Open Code Large Language Models for Code Understanding and Generation.” In: *arXiv preprint arXiv:2305.07922* (2023).
- [16] T. Wolf, L. Debut, V. Sanh, et al. “Transformers: State-of-the-Art Natural Language Processing.” In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 2020, pp. 38–45.
- [17] Y. Wu, M. Schuster, Z. Chen, et al. “Google’s Neural Machine Translation System: Bridging the Gap between Human and Machine Translation.” In: *arXiv preprint arXiv:1609.08144* (2016).